MODULE 6

RESTful API and Deployment

Building scalable endpoints, handling JSON, and deploying to production.

What is an API?

Application Programming Interface

Enables two software applications to communicate with each other.

Common Examples

- Google Maps API
- Payment Gateways (Stripe)
- Weather Data APIs



REST Architecture

Representational State Transfer

Architectural style used to design modern web APIs.

Core Principles

- ✓ Standard HTTP protocols.
- ✓ Resource-based URLs.
- ✓ JSON payload formatting.
- ✓ Standard HTTP status codes.

Resource Example

Targeting a **/students** resource:

GET	`/students`
POST	`/students`
GET	`/students/1`
PUT	`/students/1`
DELETE	`/students/1`

HTTP Methods in REST

Method	Purpose	CRUD Mapping
GET	Read or retrieve data.	READ
POST	Create new data.	CREATE
PUT / PATCH	Update existing data.	UPDATE
DELETE	Remove existing data.	DELETE

API Request and Response

Client Request

Method: GET

URL: /students

Headers: Authentication, Content-Type

Body: (Optional JSON data)



Server Response

Status Code: 200 OK

Headers: Content-Type: application/json

```
[ { "id": 1, "name": "Rahul" } ]
```

Creating Endpoints

Flask translates Python dictionaries and lists directly into JSON responses using **jsonify**.

Browser Output

```
[
  { "course": "MCA", "id": 1, "name": "Rahul" },
  { "course": "BCA", "id": 2, "name": "Priya" }
]
```

```python
from flask import Flask,jsonify

app = Flask(__name__)

@app.route("/students")
def students():
    data = [
        {"id": 1, "name": "Rahul", "course": "MCA"},
        {"id": 2, "name": "Priya", "course": "BCA"}
    ]
    return jsonify(data)

if __name__ == "__main__":
     app.run(debug=True)
```

JSON Data Exchange

JSON Object (Dictionary)

```
{ "id": 1, "name": "Rahul", "course": "MCA" }
```

JSON Array (List)

```
[
    { "id": 1, "name": "Rahul" },
    { "id": 2, "name": "Priya" }
]
```

Sending JSON from Flask

```
return jsonify(student)
```

Receiving JSON in Flask

```
data = request.get_json()
```

POST API Example

Used to receive data and create a new resource. Returns a **201 Created** status.

```
@app.route("/students", methods=["POST"])
def add_student():
     data = request.get_json()
     name = data["name"]
     course = data["course"]
     # Save to database logic here...
     return jsonify({ "message": "Student added",
"name": name, "course": course }), 201
```

Client Sends:

```
{ "name": "Amit", "course": "MCA" }
```



Server Responds:

```
{ "message": "Student added", "name": "Amit",
"course": "MCA" }
```

REST API Status Codes

2xx

Success

200 OK

201 Created

204 No Content

4xx

Client Error

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

5xx

Server Error

500 Internal Server Error

Error Handling

```
# Global 404 Handler
@app.errorhandler(404)
def not_found(error):
     return jsonify({ "error": "Resource not found"
}), 404
```

```
# Handling Invalid Data
@app.route("/students", methods=["POST"])
def add_student():
     data = request.get_json()
     if not data or "name" not in data:
          return jsonify({ "error": "Name is
          required" }), 400
     return jsonify({"message": "Added"})
```

Benefits

- Better user experience.
- Clear API responses.
- Easier debugging.
- Consistent error format.

Logging in Flask

Records important events during execution, crucial for post-deployment debugging.

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Student added successfully")
logging.warning("Invalid student data")
logging.error("Database error")
```

Level	Purpose
DEBUG	Detailed dev information.
INFO	Normal application events.
WARNING	Potential problems.
ERROR	Serious execution problem.
CRITICAL	Very serious failure.

Flask Blueprints

Organizes large Flask applications into smaller, manageable modules.

```
project/
├── app.py
├── students/
│   └── routes.py
├── users/
│   └── routes.py
└── api/
└── routes.py
```

students/routes.py

```python
from flask import Blueprint

student_bp = Blueprint("student", __name__)
@student_bp.route("/students")
def students():
     return "Student API"
```

app.py

```python
from flask import Flask
from students.routes import student_bp
app = Flask(__name__)
app.register_blueprint(student_bp)
```

Flask Deployment Flow

Development



Your Computer

Flask Dev Server

localhost:5000

Production



Hosting Server

WSGI + Flask App

Public URL

Important Pre-flight Checklist

1. Generate **requirements.txt** (pip freeze > requirements.txt).
2. Disable debug mode (Remove app.run(debug=True)).
3. Secure environment variables & database configuration.

Hosting Platforms

Render

GitHub-based automated deployment.

PythonAnywhere

Simple Python environment hosting.

Heroku

Classic Git-based cloud platform.

Module Summary

REST API › JSON › Endpoints › Error Handling › Blueprints › Deployment